

✓ Algoritmos de optimización - Seminario

Nombre y Apellidos: Lucia Sanjuan Perez

Url: https://github.com/lucissan/Algoritmos-de-optimizaci-n/blob/main/Proyecto_Algoritmos_Lucia.ipynb

Problema:

1. Sesiones de doblaje

Se precisa coordinar el doblaje de una película. Los actores del doblaje deben coincidir en las tomas en las que sus personajes aparecen juntos en las diferentes tomas. Los actores de doblaje cobran todos la misma cantidad por cada día que deben desplazarse hasta el estudio de grabación independientemente del número de tomas que se graben. No es posible grabar más de 6 tomas por día. El objetivo es planificar las sesiones por día de manera que el gasto por los servicios de los actores de doblaje sea el menor posible. Los datos son:

Número de actores: 10

Número de tomas : 30

Actores/Tomas : <https://bit.ly/36D8luK>

- 1 indica que el actor participa en la toma
- 0 en caso contrario

(*) La respuesta es obligatoria

(*)¿Cuántas posibilidades hay sin tener en cuenta las restricciones?

Si consideramos que cada toma es independiente y podría grabarse en cualquier día sin restricción, las tomas se pueden hacer todas en un día (1 posibilidad), o en cualquier cantidad de días, sin restricción.

Es decir, si solo preguntamos por la cantidad de formas en que se pueden organizar las tomas sin restricción alguna sería igual a:

$$30! = 2,65 \times 10^{32}.$$

Cada orden distinto de tomas (el orden en que se realizan) sería una posibilidad diferente.

¿Cuántas posibilidades hay teniendo en cuenta todas las restricciones.

Para calcular las posibilidades teniendo en cuenta la restricción de 6 tomas al día máximo usaremos un algoritmo de programación dinámica que calcule cuántas particiones posibles hay de 30 elementos en grupos con tamaño entre 1 y 6.

```
from math import comb

def contar_particiones(n, max_tamano):
    dp = [0] * (n + 1)
    dp[0] = 1 # Base: 1 forma de agrupar 0 tomas

    for i in range(1, n + 1):
        total = 0
        for j in range(1, min(max_tamano, i) + 1):
            total += comb(i - 1, j - 1) * dp[i - j]
        dp[i] = total
    return dp[n]

# Parámetros
numero_tomas = 30
max_tomas_por_dia = 6

# Cálculo
resultado = contar_particiones(numero_tomas, max_tomas_por_dia)
print(f"Número de formas de organizar {numero_tomas} tomas con máximo {max_tomas_por_dia} tomas por día: {resultado}")
```

Número de formas de organizar 30 tomas con máximo 6 tomas por día: 726391948970868949621309

Modelo para el espacio de soluciones

(*) ¿Cuál es la estructura de datos que mejor se adapta al problema? Argumentalo. (Es posible que hayas elegido una al principio y veas la necesidad de cambiar, argumentalo)

La estructura de datos que mejor se adapta es:

Lista de listas

dias = [

```
[0, 5, 8, 10],    # Día 1

[1, 2, 7],        # Día 2

[3, 4, 6, 9]     # Día 3
```

]

El problema consiste en agrupar tomas en días, cada sublista es una lista y cada elemento una toma.

Esta estructura permite un número variable de días, es fácil aplicar restricciones y calcular la función objetivo.

Según el modelo para el espacio de soluciones

(*)¿Cuál es la función objetivo?

(*)¿Es un problema de maximización o minimización?

La función objetivo es una función que, dada una distribución de las tomas en días, calcula el coste total de contratación de los actores. Este coste se obtiene sumando, para cada día, el número de actores distintos que participan en las tomas asignadas a dicho día, ya que cada actor cobra una vez por día independientemente del número de tomas.

El problema es de minimización, ya que el objetivo es reducir el coste total asociado a la contratación de los actores, minimizando el número de días en los que cada actor debe acudir a la grabación.

```
def coste_total(dias, data):
    total = 0

    for dia in dias: #Recorre cada día de la planificación
        #Selecciona las tomas del día y calcula qué actores participan
        actores = np.any(data[dia], axis=0)
        #Suma el número de actores distintos del día
        total += actores.sum()

    return total
```

Diseña un algoritmo para resolver el problema por fuerza bruta

Debido al elevado coste computacional del algoritmo de fuerza bruta, hemos reducido el data a 10 tomas y el máximo de días a 3 tomas por día.

```
import numpy as np

mejor_coste = float('inf')
mejor_solucion = None

data = np.array([
    [1,1,1,1,1,0,0,0,0,0],
    [0,0,1,1,1,0,0,0,0,0],
    [0,1,0,0,1,0,1,0,0,0],
    [1,1,0,0,0,0,1,1,0,0],
    [0,1,0,1,0,0,0,1,0,0],
    [1,1,0,1,1,0,0,0,0,0],
    [1,1,0,1,1,0,0,0,0,0],
    [1,1,0,0,0,1,0,0,0,0],
    [1,1,0,1,0,0,0,0,0,0],
    [1,1,0,0,0,1,0,0,1,0]
])

def backtracking(toma_actual, dias, data, max_tomas_dia):
    global mejor_coste, mejor_solucion

    if toma_actual == len(data):

        #Calculamos el coste de la solución completa
        coste = coste_total(dias, data)

        #Si es mejor que el mejor encontrado hasta ahora
        if coste < mejor_coste:
            mejor_coste = coste
            mejor_solucion = [d.copy() for d in dias]
    return
```

```

    ###Intentamos meter en días existentes
    for dia in dias:

        #Si el día no está lleno
        if len(dia) < max_tomas_dia:

            #Se crea un nuevo día con esta toma
            dia.append(toma_actual)
            #Exploramos esa rama
            backtracking(toma_actual + 1, dias, data, max_tomas_dia)
            #Deshacemos el cambio
            dia.pop()

        ###Creamos nuevo día
        dias.append([toma_actual])
        #Exploramos esa rama
        backtracking(toma_actual + 1, dias, data, max_tomas_dia)
        #Deshacemos el cambio
        dias.pop()

#Ejecución
max_tomas_dia = 3

backtracking(0, [], data, max_tomas_dia)

print("Mejor coste:", mejor_coste)
print("Mejor solución:", mejor_solucion)

```

```

Mejor coste: 19
Mejor solución: [[0, 1, 2], [3, 4, 8], [5, 6], [7, 9]]

```

Calcula la complejidad del algoritmo por fuerza bruta

El algoritmo de backtracking explora todas las posibles distribuciones de las 30 tomas en grupos de como máximo 6 tomas por día. Al generar todas las combinaciones posibles, su complejidad temporal es exponencial: $O(n)^n$

siendo n el número de tomas. La complejidad espacial es $O(n)$ debido a la recursión utilizada.

(*)Diseña un algoritmo que mejore la complejidad del algoritmo por fuerza bruta. Argumenta porque crees que mejora el algoritmo por fuerza bruta

```

import numpy as np
import random
import math

# FUNCIÓN OBJETIVO

def coste_solucion(solucion, data, max_tomas_dia):
    """Calcula el coste total de una planificación."""

    coste = 0

    # Recorre los días utilizados
    for dia in np.unique(solucion):

        # Obtiene las tomas asignadas al día
        tomas_dia = np.where(solucion == dia)[0]

        # Descarta soluciones que superen la capacidad
        if len(tomas_dia) > max_tomas_dia:
            return float("inf")

        # Obtiene los actores que trabajan ese día
        actores_dia = np.any(data[tomas_dia], axis=0)

        # Suma una asistencia por actor
        coste += actores_dia.sum()

    return int(coste)

# NORMALIZACIÓN DE LOS DÍAS

def normalizar_dias(solucion):
    """Renombra los días consecutivamente desde cero."""

    # Obtiene y ordena los días utilizados
    dias = sorted(np.unique(solucion))

```

```

    dias = sorted(np.unique(solucion))

    """ # Relaciona cada día con su nueva numeración
    equivalencias = {
        dia: nuevo_dia
        for nuevo_dia, dia in enumerate(dias)
    }

    """ # Devuelve la solución renumerada
    return np.array([
        equivalencias[dia]
        for dia in solucion
    ], dtype=int)

# CREACIÓN DE INDIVIDUOS

def crear_individuo(
    n_tomas,
    max_tomas_dia,
    max_dias
):
    """Genera una planificación aleatoria válida."""

    """ # Calcula el mínimo de días necesarios
    min_dias = math.ceil(n_tomas / max_tomas_dia)

    """ # Elige aleatoriamente el número de días
    numero_dias = random.randint(min_dias, max_dias)

    """ # Crea el vector que representa la solución
    solucion = np.empty(n_tomas, dtype=int)

    """ # Mezcla aleatoriamente las tomas
    indices = list(range(n_tomas))
    random.shuffle(indices)

    """ # Asigna una toma inicial a cada día
    for dia in range(numero_dias):
        solucion[indices[dia]] = dia

    """ # Registra la ocupación de cada día
    ocupacion = [1] * numero_dias

    """ # Asigna las tomas restantes
    for toma in indices[numero_dias:]:

        """ # Busca días con espacio disponible
        dias_disponibles = [
            dia
            for dia in range(numero_dias)
            if ocupacion[dia] < max_tomas_dia
        ]

        """ # Selecciona un día disponible
        dia = random.choice(dias_disponibles)

        """ # Asigna la toma al día
        solucion[toma] = dia

        """ # Actualiza la ocupación
        ocupacion[dia] += 1

    return normalizar_dias(solucion)

# REPARACIÓN DE INDIVIDUOS

def reparar_solucion(
    solucion,
    max_tomas_dia,
    max_dias
):
    """Corrige los días que superan la capacidad."""

    """ # Copia y normaliza la solución
    solucion = normalizar_dias(solucion.copy())

    while True:

        """ # obtiene los días utilizados
        dias = np.unique(solucion)

```

```

##### # Busca días con demasiadas tomas
##### dias_sobrecargados = [
#####     dia
#####     for dia in dias
#####     if np.sum(solucion == dia) > max_tomas_dia
##### ]

##### # Termina si la solución ya es válida
##### if not dias_sobrecargados:
#####     break

##### # elige un día sobrecargado
##### dia_origen = random.choice(dias_sobrecargados)

##### # Obtiene sus tomas
##### tomas_origen = np.where(
#####     solucion == dia_origen
##### )[0]

##### # Elige una toma para trasladarla
##### toma = random.choice(
#####     tomas_origen.tolist()
##### )

##### # Busca días con espacio disponible
##### dias_disponibles = [
#####     dia
#####     for dia in dias
#####     if dia != dia_origen
#####     and np.sum(solucion == dia) < max_tomas_dia
##### ]

##### if dias_disponibles:

#####     # Traslada la toma a otro día
#####     solucion[toma] = random.choice(
#####         dias_disponibles
#####     )

##### elif len(dias) < max_dias:

#####     # Crea un día nuevo
#####     solucion[toma] = max(dias) + 1

##### else:

#####     # Finaliza si no puede reparar la solución
#####     break

<<< return normalizar_dias(solucion)

# SELECCIÓN POR TORNEO

def seleccionar_torneo(
<<< poblacion,
<<< data,
<<< max_tomas_dia,
<<< tam_torneo=3
):
    """Selecciona el mejor de varios individuos."""

    <<< # Elige varios candidatos al azar
    <<< candidatos = random.sample(
    <<<< poblacion,
    <<<< min(tam_torneo, len(poblacion))
    <<< )

    <<< # Devuelve el candidato de menor coste
    <<< return min(
    <<<< candidatos,
    <<<< key=lambda individuo: coste_solucion(
    <<<<< individuo,
    <<<<< data,
    <<<<< max_tomas_dia
    <<<< )
    <<< ).copy()

# CRUCE

def cruzar(
<<< padre1,

```

```

    padre2,
    max_tomas_dia,
    max_dias
):
    """Combina dos soluciones para generar descendientes."""

    # Decide qué posiciones se toman de cada padre
    mascara = np.random.rand(
        len(padre1)
    ) < 0.5

    # Genera el primer descendiente
    hijo1 = np.where(
        mascara,
        padre1,
        padre2
    )

    # Genera el segundo descendiente
    hijo2 = np.where(
        mascara,
        padre2,
        padre1
    )

    # Repara el primer descendiente
    hijo1 = reparar_solucion(
        hijo1,
        max_tomas_dia,
        max_dias
    )

    # Repara el segundo descendiente
    hijo2 = reparar_solucion(
        hijo2,
        max_tomas_dia,
        max_dias
    )

    return hijo1, hijo2

```

```

# -----
# MUTACIÓN
# -----

```

```

def mutar(
    solucion,
    max_tomas_dia,
    max_dias
):
    """Aplica un cambio aleatorio a una solución."""

    # Crea una copia de la solución
    nueva = solucion.copy()

    # Elige el tipo de mutación
    tipo_mutacion = random.choice([
        "intercambio",
        "traslado",
        "nuevo_dia"
    ])

    if tipo_mutacion == "intercambio":

        # Elige dos tomas al azar
        i, j = random.sample(
            range(len(nueva)),
            2
        )

        # Intercambia sus días
        nueva[i], nueva[j] = (
            nueva[j],
            nueva[i]
        )

    elif tipo_mutacion == "traslado":

        # Elige una toma al azar
        toma = random.randrange(len(nueva))

        # Obtenga su día actual

```

```

    # Obtiene su día actual
    dia_actual = nueva[toma]

    # Busca otros días disponibles
    dias_disponibles = [
        dia
        for dia in np.unique(nueva)
        if dia != dia_actual
        and np.sum(nueva == dia) < max_tomas_dia
    ]

    if dias_disponibles:

        # Traslada la toma
        nueva[toma] = random.choice(
            dias_disponibles
        )

    elif tipo_mutacion == "nuevo_dia":

        # Obtiene los días actuales
        dias_actuales = np.unique(nueva)

        # Comprueba si se puede crear otro día
        if len(dias_actuales) < max_dias:

            # Busca tomas que puedan trasladarse
            tomas_movibles = [
                toma
                for toma in range(len(nueva))
                if np.sum(
                    nueva == nueva[toma]
                ) > 1
            ]

            if tomas_movibles:

                # Elige una toma
                toma = random.choice(
                    tomas_movibles
                )

                # Crea un nuevo día
                nueva[toma] = (
                    max(dias_actuales) + 1
                )

    # Repara y devuelve la solución
    return reparar_solucion(
        nueva,
        max_tomas_dia,
        max_dias
    )

# ALGORITMO GENÉTICO

def algoritmo_genetico(
    data,
    max_tomas_dia=6,
    max_dias=8,
    tam_poblacion=300,
    generaciones=1500,
    prob_cruce=0.90,
    prob_mutacion=0.30,
    elitismo=0.10,
    tam_torneo=3,
    semilla=42,
    mostrar_progreso=True
):

    # Fija las semillas aleatorias
    random.seed(semilla)
    np.random.seed(semilla)

    # Obtiene el número de tomas
    n_tomas = len(data)

    # Calcula el mínimo de días necesarios
    min_dias = math.ceil(
        n_tomas / max_tomas_dia
    )

```

```

""" # Comprueba que max_dias sea válido
""" if max_dias < min_dias:
"""     raise ValueError(
"""         f"Se necesitan al menos {min_dias} días."
"""     )

""" # Genera la población inicial
""" poblacion = [
"""     crear_individuo(
"""         n_tomas,
"""         max_tomas_dia,
"""         max_dias
"""     )
"""     for _ in range(tam_poblacion)
""" ]

""" # Inicializa el mejor resultado
""" mejor_solucion = None
""" mejor_coste = float("inf")

""" # Guarda la evolución del coste
""" historial = []

""" # Calcula los individuos de élite
""" numero_elite = max(
"""     1,
"""     int(tam_poblacion * elitismo)
""" )

""" # Recorre las generaciones
""" for generacion in range(generaciones):

"""     # Ordena la población por coste
"""     poblacion.sort(
"""         key=lambda individuo: coste_solucion(
"""             individuo,
"""             data,
"""             max_tomas_dia
"""         )
"""     )

"""     # Obtiene el coste del mejor individuo
"""     coste_actual = coste_solucion(
"""         poblacion[0],
"""         data,
"""         max_tomas_dia
"""     )

"""     # Actualiza la mejor solución
"""     if coste_actual < mejor_coste:

"""         mejor_coste = coste_actual
"""         mejor_solucion = poblacion[0].copy()

"""     # Guarda el mejor coste
"""     historial.append(mejor_coste)

"""     # Muestra el progreso
"""     if (
"""         mostrar_progreso
"""         and generacion % 100 == 0
"""     ):
"""         print(
"""             f"Generación {generacion}: "
"""             f"mejor coste = {mejor_coste}, "
"""             f"días = "
"""             f"{len(np.unique(mejor_solucion))}"
"""         )

"""     # Conserva los mejores individuos
"""     nueva_poblacion = [
"""         individuo.copy()
"""         for individuo
"""         in poblacion[:numero_elite]
"""     ]

"""     # Completa la nueva población
"""     while (
"""         len(nueva_poblacion)
"""         < tam_poblacion
"""     ):

```

```

##### # Selecciona el primer padre
##### padre1 = seleccionar_torneo(
#####     poblacion,
#####     data,
#####     max_tomas_dia,
#####     tam_torneo
##### )

##### # Selecciona el segundo padre
##### padre2 = seleccionar_torneo(
#####     poblacion,
#####     data,
#####     max_tomas_dia,
#####     tam_torneo
##### )

##### # Decide si aplica el cruce
##### if random.random() < probab_cruce:

#####     hijo1, hijo2 = cruzar(
#####         padre1,
#####         padre2,
#####         max_tomas_dia,
#####         max_dias
#####     )

##### else:

#####     # Conserva copias de los padres
#####     hijo1 = padre1.copy()
#####     hijo2 = padre2.copy()

#####     # Decide si muta el primer hijo
#####     if random.random() < probab_mutacion:

#####         hijo1 = mutar(
#####             hijo1,
#####             max_tomas_dia,
#####             max_dias
#####         )

#####     # Decide si muta el segundo hijo
#####     if random.random() < probab_mutacion:

#####         hijo2 = mutar(
#####             hijo2,
#####             max_tomas_dia,
#####             max_dias
#####         )

#####     # Añade el primer hijo
#####     nueva_poblacion.append(hijo1)

#####     # Añade el segundo hijo si hay espacio
#####     if (
#####         len(nueva_poblacion)
#####         < tam_poblacion
#####     ):
#####         nueva_poblacion.append(hijo2)

##### # Sustituye la población anterior
##### poblacion = nueva_poblacion

##### # Normaliza la mejor solución
##### mejor_solucion = normalizar_dias(
#####     mejor_solucion
##### )

##### # Prepara la planificación final
##### planificacion = []

##### # Recorre los días utilizados
##### for dia in sorted(
#####     np.unique(mejor_solucion)
##### ):

#####     # Obtiene las tomas del día
#####     tomas = np.where(
#####         mejor_solucion == dia
#####     )[0]

#####     # Numera las tomas desde uno
#####     planificacion.append(

```

```

def algoritmo_genetico(
    ##### (tomas + 1).tolist()
    ##### )

    return (
    ##### mejor_coste,
    ##### planificacion,
    ##### historial
    ##### )

# DATOS DEL PROBLEMA

# Cada fila representa una toma
# Cada columna representa un actor
data = np.array([
    ##### [1,1,1,1,0,0,0,0,0], # Toma 1
    ##### [0,0,1,1,1,0,0,0,0], # Toma 2
    ##### [0,1,0,0,1,0,1,0,0], # Toma 3
    ##### [1,1,0,0,0,0,1,1,0], # Toma 4
    ##### [0,1,0,1,0,0,0,1,0], # Toma 5
    ##### [1,1,0,1,1,0,0,0,0], # Toma 6
    ##### [1,1,0,1,1,0,0,0,0], # Toma 7
    ##### [1,1,0,0,0,1,0,0,0], # Toma 8
    ##### [1,1,0,1,0,0,0,0,0], # Toma 9
    ##### [1,1,0,0,0,1,0,0,1], # Toma 10
    ##### [1,1,1,0,1,0,0,1,0], # Toma 11
    ##### [1,1,1,1,0,1,0,0,0], # Toma 12
    ##### [1,0,0,1,1,0,0,0,0], # Toma 13
    ##### [1,0,1,0,0,1,0,0,0], # Toma 14
    ##### [1,1,0,0,0,0,1,0,0], # Toma 15
    ##### [0,0,0,1,0,0,0,0,1], # Toma 16
    ##### [1,0,1,0,0,0,0,0,0], # Toma 17
    ##### [0,0,1,0,0,1,0,0,0], # Toma 18
    ##### [1,0,1,0,0,0,0,0,0], # Toma 19
    ##### [1,0,1,1,1,0,0,0,0], # Toma 20
    ##### [0,0,0,0,0,1,0,1,0], # Toma 21
    ##### [1,1,1,1,0,0,0,0,0], # Toma 22
    ##### [1,0,1,0,0,0,0,0,0], # Toma 23
    ##### [0,0,1,0,0,1,0,0,0], # Toma 24
    ##### [1,1,0,1,0,0,0,0,1], # Toma 25
    ##### [1,0,1,0,1,0,0,0,1], # Toma 26
    ##### [0,0,0,1,1,0,0,0,0], # Toma 27
    ##### [1,0,0,1,0,0,0,0,0], # Toma 28
    ##### [1,0,0,0,1,1,0,0,0], # Toma 29
    ##### [1,0,0,1,0,0,0,0,0] # Toma 30
])

# EJECUCIÓN DEL ALGORITMO

# Inicializa el mejor resultado global
mejor_coste_global = float("inf")
mejor_planificacion_global = None

# Ejecuta el algoritmo con diez semillas
for semilla in range(10):

    coste, planificacion, historial = algoritmo_genetico(
    ##### data=data,
    ##### max_tomas_dia=6,
    ##### max_dias=8,
    ##### tam_poblacion=300,
    ##### generaciones=1500,
    ##### prob_cruce=0.90,
    ##### prob_mutacion=0.30,
    ##### elitismo=0.10,
    ##### tam_torneo=3,
    ##### semilla=semilla,
    ##### mostrar_progreso=False
    ##### )

    # Muestra el resultado de la ejecución
    print(
    ##### f"Ejecución {semilla + 1}: "
    ##### f"coste = {coste}, "
    ##### f"días = {len(planificacion)}"
    ##### )

    # Actualiza el mejor resultado global
    if coste < mejor_coste_global:

    ##### mejor_coste_global = coste
    ##### mejor_planificacion_global = planificacion

```

```

# -----
# RESULTADO FINAL
# -----

print("\nMEJOR RESULTADO DE TODAS LAS EJECUCIONES")

print(
    "Mejor coste encontrado:",
    mejor_coste_global
)

print(
    "Número de días:",
    len(mejor_planificacion_global)
)

# Muestra las tomas asignadas a cada día
for numero_dia, tomas in enumerate(
    mejor_planificacion_global,
    start=1
):
    print(
        f"Día {numero_dia}: {tomas} "
        f"({len(tomas)} tomas)"
    )

```

```

Ejecución 1: coste = 29, días = 5
Ejecución 2: coste = 27, días = 5
Ejecución 3: coste = 27, días = 5
Ejecución 4: coste = 28, días = 5
Ejecución 5: coste = 27, días = 5
Ejecución 6: coste = 28, días = 5
Ejecución 7: coste = 27, días = 5
Ejecución 8: coste = 27, días = 5
Ejecución 9: coste = 28, días = 5
Ejecución 10: coste = 28, días = 5

MEJOR RESULTADO DE TODAS LAS EJECUCIONES
Mejor coste encontrado: 27
Número de días: 5
Día 1: [14, 17, 18, 19, 21, 24] (6 tomas)
Día 2: [2, 20, 23, 27, 28, 30] (6 tomas)
Día 3: [1, 8, 10, 12, 26, 29] (6 tomas)
Día 4: [6, 7, 9, 13, 16, 25] (6 tomas)
Día 5: [3, 4, 5, 11, 15, 22] (6 tomas)

```

Explicación breve de cada función

- `coste_solucion()`

Calcula el coste de una planificación. Para cada día determina qué actores deben acudir y cuenta una asistencia por actor. Si algún día supera el máximo de tomas, devuelve un coste infinito para descartar la solución.

- `normalizar_dias()`

Renombra los días de forma consecutiva comenzando por cero. Esto evita representaciones diferentes de una misma planificación.

Por ejemplo:

$[2, 2, 5, 5] \rightarrow [0, 0, 1, 1]$

- `crear_individuo()`

Genera una planificación aleatoria válida. El número de días se elige entre el mínimo necesario y el máximo permitido. Después, distribuye las tomas sin superar la capacidad diaria.

- `reparar_solucion()`

Corrige los individuos no válidos generados por el cruce o la mutación. Si un día contiene demasiadas tomas, traslada algunas a días con espacio o crea un día nuevo.

- `seleccionar_torneo()`

Selecciona varios individuos aleatoriamente y devuelve el de menor coste. Este mecanismo favorece a las mejores soluciones sin eliminar por completo la diversidad.

- `cruzar()`

Combina dos planificaciones para generar dos descendientes. Cada toma hereda el día asignado por uno de los dos padres. Después, los descendientes se reparan para cumplir las restricciones.

- `mutar()`

Introduce pequeños cambios aleatorios en una planificación. Puede intercambiar dos tomas, trasladar una toma a otro día o crear un nuevo día.

- `algoritmo_genetico()`

Coordina todo el proceso. Genera la población inicial, evalúa los individuos, selecciona padres, aplica cruce y mutación, conserva los mejores y repite el proceso durante varias generaciones. Estos elementos coinciden con el funcionamiento general de los algoritmos genéticos explicado en el manual.

Explicación de los parámetros

- `max_tomas_dia`: máximo de tomas permitidas en un día.
- `max_dias`: máximo número de días explorados.
- `tam_poblacion`: número de planificaciones de cada generación.
- `generaciones`: número de iteraciones del algoritmo.
- `prob_cruce`: probabilidad de cruzar dos individuos.
- `prob_mutacion`: probabilidad de modificar un descendiente.
- `elitismo`: porcentaje de las mejores soluciones que se conservan.
- `tam_torneo`: número de individuos comparados en cada selección.
- `semilla`: permite reproducir la misma ejecución.
- `mostrar_progreso`: indica si se muestra la evolución del algoritmo.

El algoritmo se ha ejecutado diez veces utilizando semillas diferentes. Los resultados obtenidos oscilan entre 27 y 29 asistencias. La mejor planificación encontrada tiene un coste de 27 y distribuye las 30 tomas en 5 días, con 6 tomas por día. La repetición del coste 27 en varias ejecuciones muestra que el algoritmo converge de forma estable hacia soluciones de buena calidad. Sin embargo, al tratarse de un algoritmo genético, no se puede garantizar que 27 sea el óptimo global.

Se ha elegido un algoritmo genético porque el problema tiene un espacio de soluciones muy grande: existen muchas formas de distribuir 30 tomas entre distintos días, respetando un máximo de 6 tomas por día. La fuerza bruta debe explorar todas esas distribuciones, por lo que resulta computacionalmente inviable.

El algoritmo trabaja con una población de soluciones y utiliza selección, cruce, mutación y elitismo para mejorar progresivamente las planificaciones. De esta forma, obtiene una buena solución en un tiempo razonable, aunque no garantiza encontrar el óptimo global. Los algoritmos genéticos están indicados para problemas combinatorios con espacios de soluciones grandes.

(*)Calcula la complejidad del algoritmo

La complejidad temporal aproximada del algoritmo genético es $O(G \cdot P \cdot n \cdot a)$, donde G es el número de generaciones, P el tamaño de la población, n el número de tomas y a el número de actores. Si el algoritmo se ejecuta R veces con semillas diferentes, la complejidad es $O(R \cdot G \cdot P \cdot n \cdot a)$.

Al fijar el tamaño de la población, las generaciones y el número de ejecuciones, el algoritmo realiza una cantidad limitada de trabajo y evita recorrer todas las particiones posibles, como ocurre con la fuerza bruta. La complejidad espacial es $O(P \cdot n + G)$.

Por tanto, frente al crecimiento exponencial de la fuerza bruta, el algoritmo genético tiene un coste controlable mediante la población y las generaciones.

Según el problema (y tenga sentido), diseña un juego de datos de entrada aleatorios

Vamos a generar por ejemplo 20 tomas y 8 actores diferentes.

```
import numpy as np

def generar_datos(n_tomas, n_actores, min_actores=1, max_actores=5):

    data = np.zeros((n_tomas, n_actores), dtype=int)

    for i in range(n_tomas):
        # número aleatorio de actores en la toma
        num_actores = np.random.randint(min_actores, max_actores + 1)

        # elegir actores sin repetir
        actores = np.random.choice(n_actores, num_actores, replace=False)

        # asignar 1 a esos actores
        data[i, actores] = 1
```

```

    return data

data_random = generar_datos(20, 8)

print(data_random)

```

```

[[1 0 1 0 0 1 1 0]
 [1 1 0 1 1 1 0 0]
 [1 1 0 0 0 0 1 0]
 [0 0 0 1 0 0 0 1]
 [1 0 0 0 1 0 0 0]
 [0 0 0 0 1 0 0 0]
 [0 0 0 0 0 1 0 0]
 [0 1 0 0 1 1 1 0]
 [0 0 1 0 0 0 0 0]
 [0 1 1 1 0 0 1 1]
 [0 0 1 0 0 0 0 1]
 [1 0 1 0 1 1 1 0]
 [0 0 0 0 1 0 0 0]
 [1 1 1 1 0 0 0 1]
 [1 0 0 1 1 0 0 0]
 [0 0 0 1 1 0 0 0]
 [0 1 0 0 1 1 0 0]
 [1 0 1 0 1 0 1 1]
 [1 0 1 0 1 0 0 1]
 [1 0 0 0 1 0 0 0]]

```

Aplica el algoritmo al juego de datos generado

```

import numpy as np
import random
import math

# FUNCIÓN OBJETIVO

def coste_solucion(solucion, data, max_tomas_dia):
    """Calcula el coste total de una planificación."""

    coste = 0

    # Recorre los días utilizados
    for dia in np.unique(solucion):

        # Obtiene las tomas asignadas al día
        tomas_dia = np.where(solucion == dia)[0]

        # Descarta soluciones que superen la capacidad
        if len(tomas_dia) > max_tomas_dia:
            return float("inf")

        # Obtiene los actores que trabajan ese día
        actores_dia = np.any(data[tomas_dia], axis=0)

        # Suma una asistencia por actor
        coste += actores_dia.sum()

    return int(coste)

# NORMALIZACIÓN DE LOS DÍAS

def normalizar_dias(solucion):
    """Renombra los días consecutivamente desde cero."""

    # Obtiene y ordena los días utilizados
    dias = sorted(np.unique(solucion))

    # Relaciona cada día con su nueva numeración
    equivalencias = {
        dia: nuevo_dia
        for nuevo_dia, dia in enumerate(dias)
    }

    # Devuelve la solución renumerada
    return np.array([
        equivalencias[dia]
        for dia in solucion
    ], dtype=int)

# CREACIÓN DE INDIVIDUOS

```

```

def crear_individuo(
    n_tomas,
    max_tomas_dia,
    max_dias
):
    """Genera una planificación aleatoria válida."""

    # Calcula el mínimo de días necesarios
    min_dias = math.ceil(n_tomas / max_tomas_dia)

    # Elige aleatoriamente el número de días
    numero_dias = random.randint(min_dias, max_dias)

    # Crea el vector que representa la solución
    solucion = np.empty(n_tomas, dtype=int)

    # Mezcla aleatoriamente las tomas
    indices = list(range(n_tomas))
    random.shuffle(indices)

    # Asigna una toma inicial a cada día
    for dia in range(numero_dias):
        solucion[indices[dia]] = dia

    # Registra la ocupación de cada día
    ocupacion = [1] * numero_dias

    # Asigna las tomas restantes
    for toma in indices[numero_dias:]:

        # Busca días con espacio disponible
        dias_disponibles = [
            dia
            for dia in range(numero_dias)
            if ocupacion[dia] < max_tomas_dia
        ]

        # Selecciona un día disponible
        dia = random.choice(dias_disponibles)

        # Asigna la toma al día
        solucion[toma] = dia

        # Actualiza la ocupación
        ocupacion[dia] += 1

    return normalizar_dias(solucion)

# REPARACIÓN DE INDIVIDUOS

def reparar_solucion(
    solucion,
    max_tomas_dia,
    max_dias
):
    """Corrige los días que superan la capacidad."""

    # Copia y normaliza la solución
    solucion = normalizar_dias(solucion.copy())

    while True:

        # obtiene los días utilizados
        dias = np.unique(solucion)

        # Busca días con demasiadas tomas
        dias_sobrecargados = [
            dia
            for dia in dias
            if np.sum(solucion == dia) > max_tomas_dia
        ]

        # Termina si la solución ya es válida
        if not dias_sobrecargados:
            break

        # elige un día sobrecargado
        dia_origen = random.choice(dias_sobrecargados)

        # Obtiene sus tomas
        tomas_origen = np.where(

```

```

        solucion == dia_origen
    )[0]

    # Elige una toma para trasladarla
    toma = random.choice(
        tomas_origen.tolist()
    )

    # Busca días con espacio disponible
    dias_disponibles = [
        dia
        for dia in dias
        if dia != dia_origen
        and np.sum(solucion == dia) < max_tomas_dia
    ]

    if dias_disponibles:

        # Traslada la toma a otro día
        solucion[toma] = random.choice(
            dias_disponibles
        )

    elif len(dias) < max_dias:

        # Crea un día nuevo
        solucion[toma] = max(dias) + 1

    else:

        # Finaliza si no puede reparar la solución
        break

    return normalizar_dias(solucion)

# SELECCIÓN POR TORNEO
def seleccionar_torneo(
    poblacion,
    data,
    max_tomas_dia,
    tam_torneo=3
):
    """Selecciona el mejor de varios individuos."""

    # Elige varios candidatos al azar
    candidatos = random.sample(
        poblacion,
        min(tam_torneo, len(poblacion))
    )

    # Devuelve el candidato de menor coste
    return min(
        candidatos,
        key=lambda individuo: coste_solucion(
            individuo,
            data,
            max_tomas_dia
        )
    ).copy()

# CRUCE
def cruzar(
    padre1,
    padre2,
    max_tomas_dia,
    max_dias
):
    """Combina dos soluciones para generar descendientes."""

    # Decide qué posiciones se toman de cada padre
    mascara = np.random.rand(
        len(padre1)
    ) < 0.5

    # Genera el primer descendiente
    hijo1 = np.where(
        mascara,
        padre1,
        padre2
    )

```

```

    paurez
)

# Genera el segundo descendiente
hijo2 = np.where(
    mascara,
    padre2,
    padre1
)

# Repara el primer descendiente
hijo1 = reparar_solucion(
    hijo1,
    max_tomas_dia,
    max_dias
)

# Repara el segundo descendiente
hijo2 = reparar_solucion(
    hijo2,
    max_tomas_dia,
    max_dias
)

return hijo1, hijo2

# -----
# MUTACIÓN
# -----

def mutar(
    solucion,
    max_tomas_dia,
    max_dias
):
    """Aplica un cambio aleatorio a una solución."""

    # Crea una copia de la solución
    nueva = solucion.copy()

    # Elige el tipo de mutación
    tipo_mutacion = random.choice([
        "intercambio",
        "traslado",
        "nuevo_dia"
    ])

    if tipo_mutacion == "intercambio":

        # Elige dos tomas al azar
        i, j = random.sample(
            range(len(nueva)),
            2
        )

        # Intercambia sus días
        nueva[i], nueva[j] = (
            nueva[j],
            nueva[i]
        )

    elif tipo_mutacion == "traslado":

        # Elige una toma al azar
        toma = random.randrange(len(nueva))

        # Obtiene su día actual
        dia_actual = nueva[toma]

        # Busca otros días disponibles
        dias_disponibles = [
            dia
            for dia in np.unique(nueva)
            if dia != dia_actual
            and np.sum(nueva == dia) < max_tomas_dia
        ]

        if dias_disponibles:

            # Traslada la toma
            nueva[toma] = random.choice(
                dias_disponibles
            )

```

```

    )

    elif tipo_mutacion == "nuevo_dia":

        # Obtiene los días actuales
        dias_actuales = np.unique(nueva)

        # Comprueba si se puede crear otro día
        if len(dias_actuales) < max_dias:

            # Busca tomas que puedan trasladarse
            tomas_movibles = [
                toma
                for toma in range(len(nueva))
                if np.sum(
                    nueva == nueva[toma]
                ) > 1
            ]

            if tomas_movibles:

                # Elige una toma
                toma = random.choice(
                    tomas_movibles
                )

                # Crea un nuevo día
                nueva[toma] = (
                    max(dias_actuales) + 1
                )

            # Repara y devuelve la solución
            return reparar_solucion(
                nueva,
                max_tomas_dia,
                max_dias
            )

# ALGORITMO GENÉTICO

def algoritmo_genetico(
    data,
    max_tomas_dia=6,
    max_dias=8,
    tam_poblacion=300,
    generaciones=1500,
    prob_cruce=0.90,
    prob_mutacion=0.30,
    elitismo=0.10,
    tam_torneo=3,
    semilla=42,
    mostrar_progreso=True
):

    # Fija las semillas aleatorias
    random.seed(semilla)
    np.random.seed(semilla)

    # Obtiene el número de tomas
    n_tomas = len(data)

    # Calcula el mínimo de días necesarios
    min_dias = math.ceil(
        n_tomas / max_tomas_dia
    )

    # Comprueba que max_dias sea válido
    if max_dias < min_dias:
        raise ValueError(
            f"Se necesitan al menos {min_dias} días."
        )

    # Genera la población inicial
    poblacion = [
        crear_individuo(
            n_tomas,
            max_tomas_dia,
            max_dias
        )
        for _ in range(tam_poblacion)
    ]

```

```
# Inicializa el mejor resultado
mejor_solucion = None
mejor_coste = float("inf")

# Guarda la evolución del coste
historial = []

# Calcula los individuos de élite
numero_elite = max(
    1,
    int(tam_poblacion * elitismo)
)

# Recorre las generaciones
for generacion in range(generaciones):

    # Ordena la población por coste
    poblacion.sort(
        key=lambda individuo: coste_solucion(
            individuo,
            data,
            max_tomas_dia
        )
    )

    # Obtiene el coste del mejor individuo
    coste_actual = coste_solucion(
        poblacion[0],
        data,
        max_tomas_dia
    )

    # Actualiza la mejor solución
    if coste_actual < mejor_coste:

        mejor_coste = coste_actual
        mejor_solucion = poblacion[0].copy()

    # Guarda el mejor coste
    historial.append(mejor_coste)

    # Muestra el progreso
    if (
        mostrar_progreso
        and generacion % 100 == 0
    ):
        print(
            f"Generación {generacion}: "
            f"mejor coste = {mejor_coste}, "
            f"días = "
            f"{len(np.unique(mejor_solucion))}"
        )

    # Conserva los mejores individuos
    nueva_poblacion = [
        individuo.copy()
        for individuo
        in poblacion[:numero_elite]
    ]

    # Completa la nueva población
    while (
        len(nueva_poblacion)
        < tam_poblacion
    ):

        # Selecciona el primer padre
        padre1 = seleccionar_torneo(
            poblacion,
            data,
            max_tomas_dia,
            tam_torneo
        )

        # Selecciona el segundo padre
        padre2 = seleccionar_torneo(
            poblacion,
            data,
            max_tomas_dia,
            tam_torneo
        )
```

```

# Decide si aplica el cruce
if random.random() < probab_cruce:

    hijo1, hijo2 = cruzar(
        padre1,
        padre2,
        max_tomas_dia,
        max_dias
    )

else:

    # Conserva copias de los padres
    hijo1 = padre1.copy()
    hijo2 = padre2.copy()

# Decide si muta el primer hijo
if random.random() < probab_mutacion:

    hijo1 = mutar(
        hijo1,
        max_tomas_dia,
        max_dias
    )

# Decide si muta el segundo hijo
if random.random() < probab_mutacion:

    hijo2 = mutar(
        hijo2,
        max_tomas_dia,
        max_dias
    )

# Añade el primer hijo
nueva_poblacion.append(hijo1)

# Añade el segundo hijo si hay espacio
if (
    len(nueva_poblacion)
    < tam_poblacion
):
    nueva_poblacion.append(hijo2)

# Sustituye la población anterior
poblacion = nueva_poblacion

# Normaliza la mejor solución
mejor_solucion = normalizar_dias(
    mejor_solucion
)

# Prepara la planificación final
planificacion = []

# Recorre los días utilizados
for dia in sorted(
    np.unique(mejor_solucion)
):

    # Obtiene las tomas del día
    tomas = np.where(
        mejor_solucion == dia
    )[0]

    # Numera las tomas desde uno
    planificacion.append(
        (tomas + 1).tolist()
    )

return (
    mejor_coste,
    planificacion,
    historial
)

# EJECUCIÓN DEL ALGORITMO con DATA_RANDOM

# Inicializa el mejor resultado global
mejor_coste_global = float("inf")
mejor_planificacion_global = None

```

```

# Ejecuta el algoritmo con diez semillas
for semilla in range(10):

    coste, planificacion, historial = algoritmo_genetico(
        data=data_random,
        max_tomas_dia=6,
        max_dias=8,
        tam_poblacion=300,
        generaciones=1500,
        prob_cruce=0.90,
        prob_mutacion=0.30,
        elitismo=0.10,
        tam_torneo=3,
        semilla=semilla,
        mostrar_progreso=False
    )

    # Muestra el resultado de la ejecución
    print(
        f"Ejecución {semilla + 1}: "
        f"coste = {coste}, "
        f"días = {len(planificacion)}"
    )

    # Actualiza el mejor resultado global
    if coste < mejor_coste_global:

        mejor_coste_global = coste
        mejor_planificacion_global = planificacion

# -----
# RESULTADO FINAL
# -----

print("\nMEJOR RESULTADO DE TODAS LAS EJECUCIONES")

print(
    "Mejor coste encontrado:",
    mejor_coste_global
)

print(
    "Número de días:",
    len(mejor_planificacion_global)
)

# Muestra las tomas asignadas a cada día
for numero_dia, tomas in enumerate(
    mejor_planificacion_global,
    start=1
):
    print(
        f"Día {numero_dia}: {tomas} "
        f"({len(tomas)} tomas)"
    )

```

```

Ejecución 1: coste = 19, días = 4
Ejecución 2: coste = 19, días = 4
Ejecución 3: coste = 19, días = 4
Ejecución 4: coste = 19, días = 4
Ejecución 5: coste = 19, días = 4
Ejecución 6: coste = 19, días = 4
Ejecución 7: coste = 19, días = 4
Ejecución 8: coste = 19, días = 4
Ejecución 9: coste = 20, días = 4
Ejecución 10: coste = 19, días = 4

MEJOR RESULTADO DE TODAS LAS EJECUCIONES
Mejor coste encontrado: 19
Número de días: 4
Día 1: [4, 10, 11, 14, 18, 19] (6 tomas)
Día 2: [1, 3, 5, 8, 9, 12] (6 tomas)
Día 3: [6, 13] (2 tomas)
Día 4: [2, 7, 15, 16, 17, 20] (6 tomas)

```

Describe brevemente las líneas de cómo crees que es posible avanzar en el estudio del problema. Ten en cuenta incluso posibles variaciones del problema y/o variaciones al alza del tamaño

